

판다스 (Pandas) (2)

2026-1학기

ICT융합학부 컴퓨터공학트랙

김대환

목차

1. Pandas에서 데이터 연산
2. 누락된 데이터 처리하기

Pandas에서 데이터 연산

Pandas에서 데이터 연산 (1)

■ Pandas에서 데이터 연산

- 기본 산술 연산 (덧셈, 뺄셈, 곱셈 등)과 복잡한 연산(삼각함수, 지수와 로그 함수등) 모두에서 요소 단위의 연산을 빠르게 수행 가능
- 단항 연산: 유니버설 함수가 결과물에 인덱스와 열 레이블 보존
- 이항 연산: 유니버설 함수에 게채를 전달할 때 자동으로 인덱스를 정렬

Python OperatorPandas Method(s)

+	add()
-	sub(), subtract()
*	mul(), multiply()
/	truediv(), div(), divide()
//	floordiv()
%	mod()
**	pow()

Pandas에서 데이터 연산 (2)

- 유니버설 함수: 인덱스 보존

- Series와 DataFrame 객체에 유니버설 함수 동작

```
import pandas as pd
import numpy as np
```

```
rng = np.random.RandomState(42)
ser = pd.Series(rng.randint(0, 10, 4))
ser
```

```
0    6
1    3
2    7
3    4
dtype: int64
```

```
df = pd.DataFrame(rng.randint(0, 10, (3, 4)),
                  columns=['A', 'B', 'C', 'D'])
df
```

	A	B	C	D
0	6	9	2	6
1	7	4	3	7
2	7	2	5	4

```
np.exp(ser)
```

```
0    403.428793
1     20.085537
2   1096.633158
3     54.598150
dtype: float64
```

인덱스 그대로 보존

```
np.sin(df * np.pi / 4)
```

	A	B	C	D
0	-1.000000	7.071068e-01	1.000000	-1.000000e+00
1	-0.707107	1.224647e-16	0.707107	-7.071068e-01
2	-0.707107	1.000000e+00	-0.707107	1.224647e-16

Pandas에서 데이터 연산 (3)

■ 유니버설 함수: 인덱스 정렬

- 2개의 Series 또는 DataFrame 객체에 이항 연산을 적용 → 연산 수행 시 인덱스를 정렬

■ (예시) Series에서 인덱스 정렬

```
area = pd.Series({'Alaska': 1723337, 'Texas': 695662,
                  'California': 423967}, name='area')
population = pd.Series({'California': 38332521, 'Texas': 26448193,
                       'New York': 19651127}, name='population')
```

population / area

Alaska	NaN
California	90.413926
New York	NaN
Texas	38.018740

dtype: float64

두 배열 인덱스의 합집합

```
area.index | population.index
```

Index(['Alaska', 'California', 'New York', 'Texas'], dtype='object')

```
A = pd.Series([2, 4, 6], index=[0, 1, 2])
B = pd.Series([1, 3, 5], index=[1, 2, 3])
A + B
```

0	NaN
1	5.0
2	9.0
3	NaN

dtype: float64

둘 중에 하나라도 없는 항목
→ NaN (Not a Number)

```
A.add(B, fill_value=0)
```

0	2.0
1	5.0
2	9.0
3	5.0

dtype: float64

NaN 원치 않으면, 누락된
요소 값 명시적 입력 가능
→ B[0]=0, A[3]=0 입력

Pandas에서 데이터 연산 (4)

- DataFrame에서 인덱스 정렬

- 결과 인덱스 모두에서 정렬 발생

```
A = pd.DataFrame(rng.randint(0, 20, (2, 2)),  
                  columns=list('AB'))
```

A

	A	B
0	1	1
1	5	1

```
B = pd.DataFrame(rng.randint(0, 10, (3, 3)),  
                  columns=list('BAC'))
```

B

	B	A	C
0	4	0	9
1	5	8	0
2	9	2	6

A + B

	A	B	C
0	1.0	15.0	NaN
1	13.0	6.0	NaN
2	NaN	NaN	NaN

← 두 객체의 순서에 상관없이
결과 인덱스 정렬

```
fill = A.stack().mean()  
A.add(B, fill_value=fill)
```

	A	B	C
0	1.0	15.0	13.5
1	13.0	6.0	4.5
2	6.5	13.5	10.5

← NaN 원치 않으면, 누락된
요소 값 명시적 입력 가능

Pandas에서 데이터 연산 (5)

- 유니버설함수: DataFrame과 Series 간 연산
 - 인덱스와 열의 순서는 비슷하게 유지

```
A = rng.randint(10, size=(3, 4))  
A
```

```
array([[3, 8, 2, 4],  
       [2, 6, 4, 8],  
       [6, 1, 3, 8]])
```

```
A - A[0] 행 방향 적용
```

```
array([[ 0,  0,  0,  0],  
       [-1, -2,  2,  4],  
       [ 3, -7,  1,  4]])
```

```
df = pd.DataFrame(A, columns=list('QRST'))  
df - df.iloc[0]
```

	Q	R	S	T
0	0	0	0	0
1	-1	-2	2	4
2	3	-7	1	4

```
df.subtract(df['R'], axis=0)
```

	Q	R	S	T
0	5	0	-6	-4
1	4	0	-2	2
2	5	0	2	7

연산 진행하지 않는 R, T 열은 NaN

```
halfrow = df.iloc[0, ::2]  
halfrow
```

```
Q    3  
S    2
```

Name: 0, dtype: int64

```
df - halfrow
```

	Q	R	S	T
0	0.0	NaN	0.0	NaN
1	-1.0	NaN	2.0	NaN
2	3.0	NaN	1.0	NaN

누락된 데이터 처리하기

누락된 데이터 처리하기 (1)

- 누락된 데이터 처리 방식 2가지
 - 마스킹 방식
 - 마스크는 완전히 별개의 부울 배열 방식 or
 - 지역적으로 값의 널 상태를 가리키기 위해 데이터 표현에서 1비트를 전용으로 사용
 - 스토리지와 연산에서 오버헤드 존재
 - 센티널 방식
 - 누락된 정숫값을 -9999나 보기 드문 비트 패턴으로 표시 or
 - 부동 소수점 값을 IEEE 부동 소수점 표준을 따르는 특수 값인 NaN (Not a Number)로 표시
 - 유효값의 범위가 줄어들고, 별도의 연산 로직이 필요

누락된 데이터 처리하기 (2)

- **Pandas에서 누락된 데이터**
 - 널 값을 저장하고 조작하는 2가지 모드 존재
 - (기본 모드) 센티널 기반 누락 데이터 체계를 사용, 데이터 타입에 따라 센티널 값이 **NaN** 또는 **None**으로 설정
 - (다른 모드) 널 값이 들어갈 수 있는 데이터 타입(**dtypes**)를 사용, 이 경우 마스크 배열이 함께 생성되어 누락된 항목을 추적. 이렇게 누락된 항목은 특수한 **pd.NA** 값으로 사용자에게 표시

누락된 데이터 처리하기 (3)

▪ Pandas에서 누락된 데이터 (2)

- None: 센티널 값
 - None을 센티널 값으로 사용
 - None은 파이썬 객체 → None을 포함하는 모든 배열이 dtype=object를 가지고 있어야 함

```
import numpy as np
import pandas as pd
```

```
vals1 = np.array([1, None, 3, 4])
vals1
```

```
array([1, None, 3, 4], dtype=object)
```

```
for dtype in ['object', 'int']:
    print("dtype =", dtype)
    %timeit np.arange(1E6, dtype=dtype).sum()
    print()
```

```
dtype = object
10 loops, best of 3: 78.2 ms per loop
```

```
dtype = int
100 loops, best of 3: 3.06 ms per loop
```

많은 오버헤드로
연산이 느림!!

산술 연산 지원 불가 !!

```
vals1.sum()
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-4-749fd8ae6030> in <module>()
----> 1 vals1.sum()
```

```
/Users/jakevdp/anaconda/lib/python3.5/site-packages/numpy/core/numeric.py:30
31 def _sum(a, axis=None, dtype=None, out=None, keepdims=False):
--> 32     return umr_sum(a, axis, dtype, out, keepdims)
33
34 def _prod(a, axis=None, dtype=None, out=None, keepdims=False):
```

```
TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'
```

누락된 데이터 처리하기 (4)

▪ Pandas에서 누락된 데이터 (3)

▪ NaN: 누락된 숫자 데이터

- NaN (Not a Number)
- IEEE 부동 소수점 표기를 사용하는 모든 시스템이 인식하는 특수 부동 소수점 값 ([빠른 연산 지원](#))

```
vals2 = np.array([1, np.nan, 3, 4])  
vals2.dtype  
  
dtype('float64')
```

```
1 + np.nan
```

nan

```
0 * np.nan
```

nan

```
vals2.sum(), vals2.min(), vals2.max()  
  
(nan, nan, nan)
```

누락된 값을 무시하고 계산 진행

```
np.nansum(vals2), np.nanmin(vals2), np.nanmax(vals2)  
  
(8.0, 1.0, 4.0)
```

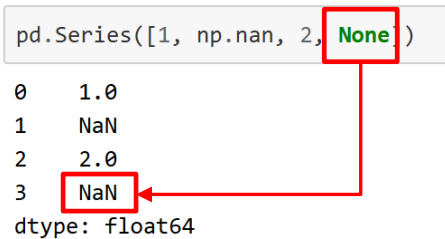
누락된 데이터 처리하기 (5)

■ Pandas에서 누락된 데이터 (4)

■ Pandas에서 NaN과 None

- NaN과 None은 서로 역할 존재
- 둘을 거의 호환성 있게 처리하고, 적절한 경우에는 서로 변환 가능

```
pd.Series([1, np.nan, 2, None])
```



```
0    1.0
1    NaN
2    2.0
3    NaN
dtype: float64
```

- 사용할 수 있는 센티널 값이 없는 타입인 경우 → NA 값이 있으면 자동으로 타입 변환

```
x = pd.Series(range(2), dtype=int)
x
```

```
0    0
1    1
dtype: int64
```

```
x[0] = None
x
```

→ 정수 배열 값을 np.nan 설정하면 → 부동 소수점 타입으로 자동 상향 변환

```
0    NaN
1    1.0
dtype: float64
```

→ 자동으로 None을 NaN으로 변경

누락된 데이터 처리하기 (6)

■ 널 값 연산하기 (1)

- None과 NaN을 누락된 값이나 널 값을 가리키기 위해 호환되는 값으로 처리
- 널 값을 감지하고 삭제하고 대체하는 메소드들
 - **isnull()**: 누락 값을 가리키는 부울 마스크를 생성
 - **notnull()**: isnull()의 역
 - **dropna()**: 데이터에 필터를 적용한 버전을 반환
 - **fillna()**: 누락 값을 채우거나 전가된 데이터 사본을 반환

누락된 데이터 처리하기 (7)

- 널 값 연산하기 (2)

- 널 값 탐지

- `isnull()`, `notnull()`

- 둘 다 데이터에 대한 부울 마스크를 반환

```
data = pd.Series([1, np.nan, 'hello', None])
```

```
data.isnull()
```

```
0    False
1     True
2    False
3     True
dtype: bool
```

```
data[data.notnull()]
```

```
0     1
2  hello
dtype: object
```


누락된 데이터 처리하기 (8)

- 널 값 연산하기 (3)
 - 널 값 제거하기
 - **dropna()**: NA 값 제거하기
 - **fillna()**: NA 값 채우기

```
data.dropna()
```

```
0      1
2  hello
dtype: object
```

```
df = pd.DataFrame([[1,      np.nan, 2],
                   [2,      3,      5],
                   [np.nan, 4,      6]])
df
```

	0	1	2
0	1.0	NaN	2
1	2.0	3.0	5
2	NaN	4.0	6

널 값이 있는 모든 행 삭제

```
df.dropna()
```

	0	1	2
1	2.0	3.0	5

널 값이 있는 모든 열 삭제

```
df.dropna(axis='columns')
```

	2
0	2
1	5
2	6

누락된 데이터 처리하기 (9)

■ 널 값 연산하기 (4)

■ 널 값 제거하기 (cont')

- 모두 NA 값으로 채워져 있거나 NA 값이 대부분을 차지하는 행이나 열을 삭제하고 싶을 때
 - how나 thresh 매개변수 사용
 - how='any' : 널 값을 포함하는 행이나 열을 모두 삭제
 - how='all' : 모두 널 값인 행이나 열만 삭제

```
df[3] = np.nan  
df
```

	0	1	2	3
0	1.0	NaN	2	NaN
1	2.0	3.0	5	NaN
2	NaN	4.0	6	NaN

```
df.dropna(axis='rows', thresh=3)
```

	0	1	2	3
1	2.0	3.0	5	NaN

```
df.dropna(axis='columns', how='all')
```

	0	1	2
0	1.0	NaN	2
1	2.0	3.0	5
2	NaN	4.0	6

누락된 데이터 처리하기 (10)

■ 널 값 연산하기 (5)

■ 널 값 채우기

- 널 값을 유효한 값으로 대체 필요
- 널 값을 대체한 배열의 사본을 반환하는 `fillna()` 메서드 제공

```
data = pd.Series([1, np.nan, 2, None, 3], index=list('abcde'))  
data
```

```
a    1.0  
b    NaN  
c    2.0  
d    NaN  
e    3.0  
dtype: float64
```

```
data.fillna(0)
```

```
a    1.0  
b    0.0  
c    2.0  
d    0.0  
e    3.0  
dtype: float64
```

```
# forward-fill
```

```
data.fillna(method='ffill')
```

```
a    1.0  
b    1.0  
c    2.0  
d    2.0  
e    3.0  
dtype: float64
```

```
# back-fill
```

```
data.fillna(method='bfill')
```

```
a    1.0  
b    2.0  
c    2.0  
d    3.0  
e    3.0  
dtype: float64
```

```
df
```

	0	1	2	3
0	1.0	NaN	2	NaN
1	2.0	3.0	5	NaN
2	NaN	4.0	6	NaN

```
df.fillna(method='ffill', axis=1)
```

	0	1	2	3
0	1.0	1.0	2.0	2.0
1	2.0	3.0	5.0	5.0
2	NaN	4.0	6.0	6.0

이전 값으로 채울 수 없다면,
그냥 NaN으로 남음

Q & A
